Smart Parking System

An IoT Architecture for Occupancy Monitoring, Allocation and Short-Horizon Forecasting

Akil
Registration number: 25BCE2777
Institution: [Enter institution]
Course: [Enter course]
Faculty / guide: [Enter name]
Final project report · 20 October 2026

Abstract

Parking guidance requires occupancy, reliable vehicle records, and a rule for choosing among free spaces. This project presents a five-space smart-parking prototype integrating RFID session handling, camera occupancy processing, slot allocation, short-horizon forecasting, and a browser dashboard. A FastAPI service and SQLite database coordinate state. ESP32 controller firmware reads RFID tags and requests entry or exit decisions; ESP32-CAM firmware serves JPEG captures to a Python ingestion process. Forecast models run on the server, keeping neural inference outside the microcontrollers. Evaluation uses a synthetic timeline of 4,032 five-minute intervals with 27 features. LSTM, GRU, and temporal Transformer regressors share a 12-step lookback, one-step horizon, chronological 70/15/15 split, and training-only scaling. On the synthetic test partition, GRU obtains MAE 0.098055 and RMSE 0.117862; persistence obtains MAE 0.120875 and RMSE 0.169173. The allocation study evaluates five policies across 120 scenarios with three seeded repetitions. Exact enumeration and the genetic algorithm both achieve mean objective cost 0.378194; PSO and ACO have mean optimality gaps 0.048264 and 0.003586. These values describe generated observations and five-slot software simulations, not field performance. Automated tests cover lifecycle, camera input, occupancy transitions, and prediction behavior; both firmware targets have compile evidence. The user reports that the assembled tabletop hardware works; this qualitative report was not captured as a serial trial log. Earlier RC522 diagnostics recorded invalid SPI responses before that report. RFID success rates, camera accuracy, OLED/LED correctness, servo timing, and power remain without measured trials. The camera detector is an edge-density heuristic requiring calibrated regions and real-scene validation. This report documents architecture, proposed wiring, implementation, measurements, limitations, and bench procedures. The prototype demonstrates an auditable software integration path; deployment conclusions require real data, occupancy labels, electrical checks, and supervised hardware trials.

Keywords: smart parking, Internet of Things, RFID, camera occupancy, allocation optimization, occupancy forecasting, LSTM, GRU, Transformer

1. Introduction

Parking search is a coordination problem: drivers need current availability, operators need a consistent record of arrivals and departures, and a guidance policy must select from the spaces that remain free. Sensors and networked systems can make that state visible, but the quality of a result depends on the sensing method, communication reliability, and whether the allocation and forecasting methods are evaluated against appropriate baselines [1].

This project separates device I/O from server computation. An ESP32 controller reads an RFID tag and calls the existing vehicle lifecycle API. A camera module can provide JPEG snapshots to a Python polling service; that service forwards valid frames to the backend's existing occupancy endpoint. FastAPI coordinates persistence, occupancy, allocation, prediction, and the browser dashboard. TensorFlow/Keras forecasting runs on the server; neither the ESP32 controller nor the ESP32-CAM performs neural inference.

The software contributions are (i) an idempotent API lifecycle and shared allocation objective, (ii) a privacy-conscious simulated time-series dataset and comparable neural forecasting experiment, (iii) a reproducible GA/PSO/ACO benchmark with deterministic and exact references, and (iv) a hardware integration path that fails closed when allocation cannot be confirmed. The user reports that the assembled tabletop hardware works. Since no post-rewire trial logs or quantitative measurements were captured, the report distinguishes that qualitative user report from measured results and provides only source/specification-based engineering estimates where defensible.

The Arduino framework on the ESP32 is suited to deterministic sensor polling and actuator control, while the host computer provides the memory and processing environment for training and running the neural regressors. This split keeps the microcontroller responsible for bounded I/O and authorization checks and leaves data-intensive inference on the server.

2. Literature Survey

Prior work spans sensing, RFID-mediated access, vision datasets, forecasting, and allocation. The comparison below distinguishes the evaluated scope of each source from the narrower five-slot simulated experiments in this project.

Table 1. Related studies and their relevance to this project

Study	Venue	Method / reported contribution	Relevance and limitation
de Almeida et al. [14]	2022 · Expert Systems with Applications	Systematic review of public datasets and computer-vision methods for parking-lot management.	Finds dataset dependence and weak evaluation protocols; motivates explicit limits on this project's heuristic and absence of labeled field images.
Lee et al. [15]	2022 · Microprocessors and Microsystems	Reviews edge-enabled smart parking with emphasis on security, energy use, sensors, and communication.	Highlights security and energy considerations; this prototype's HTTP endpoints remain unauthenticated and its power is unmeasured.
Feng et al. [16]	2022 · Complex & Intelligent Systems	Uses a dual ConvLSTM and dense convolution network for zone-wise availability prediction over public-lot data.	Models spatial relationships across lots and reports real-data experiments; the present five-slot regression uses synthetic data and no spatial graph.
Zhang et al. [17]	2024 · ISPRS International Journal of Geo-Information	Compares TCN and LSTM and proposes MAT-LSTM for parking-space availability using real and synthetic datasets.	Supports controlled comparisons across data sources; it does not validate the architectures or synthetic generator used in this project.
Enríquez et al. [18]	2024 · Applied System Innovation	Studies AdaBoost parking prediction with real parking-flow data and simulated fog/cloud latency.	Shows the distinction between measured prediction and simulated network latency; this project reports server-side model timing only.

Recent work emphasizes dataset-independent vision evaluation [14], security and energy concerns in edge deployments [15], spatial forecasting across zones [16], recurrent and convolutional model comparisons [17], and prediction with fog/cloud architectures [18]. Foundational model formulations used here are LSTM [6], GRU [7], and self-attention [8]. These publications define methods and identify evaluation needs; they do not validate this project's synthetic corpus or hardware. The gap addressed here is a small, reproducible integration with matched model inputs and explicit simulation limits, rather than a new state-of-the-art claim.

Across these strands, prior work commonly emphasizes a particular sensing modality, city-scale prediction task, or optimization formulation. The present contribution is an integrated, inspectable prototype and a controlled comparison on its five-slot objective, with exact enumeration used to test whether metaheuristics add value at this scale. The contribution is engineering and evaluation of this system instance; it is not a new optimization algorithm or a field-validated occupancy model.

3. Problem Statement and Objectives

The system must keep a consistent record of which configured parking spaces are occupied, associate a vehicle's entry and exit with idempotent operations, select only a free slot, and expose current and short-horizon occupancy through a dashboard/API. A camera-based heuristic may suggest occupancy transitions, but its output is confidence-filtered and temporally smoothed before persistence. A forecast is a regression estimate over aggregated server history, not a direct image classifier.

Table 2. Objectives, completion status, and available evidence

Objective	Status and evidence
Identify configured-slot occupancy	Software implemented: JPEG ROI edge-density detector, confidence filter, consecutive-frame smoothing, SQLite persistence. Real-camera accuracy is not measured.
RFID entry/exit	Backend API complete and tested; controller firmware implements polling, idempotent retry and response validation. Physical reader tests remain open.
Compare GA, PSO, ACO allocation	Complete: 120 scenarios, three repeat seeds, common objective, first-available and exhaustive references.
Forecast next-interval occupancy	Complete on deterministic synthetic data using LSTM, GRU, Transformer and persistence baseline.
Dashboard visibility	Existing dashboard and API are implemented; browser screenshot and physical-device indicators were not captured in this environment.
Low-cost hardware integration	Firmware projects compile and connection design is documented. A generic CH340 adapter was detected but its target is unknown; no peripheral behavior is verified.

4. System Architecture

The backend remains the system coordinator. Camera capture and RFID access use distinct flows so a camera outage does not invent a vehicle operation and a network failure cannot trigger a gate opening. SQLite is the local system of record. The dashboard consumes backend routes rather than reading device state directly.

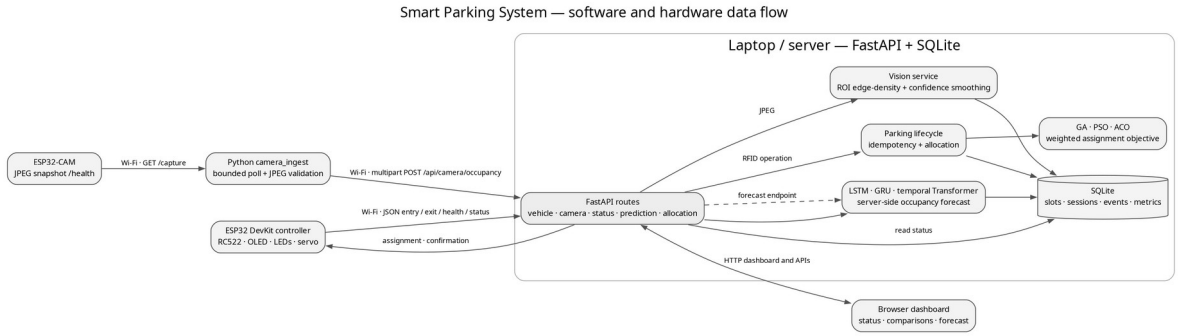


Figure 1. System data flow; compute-intensive vision aggregation, allocation, forecasting and persistence run on the server.

Camera flow: ESP32-CAM GET /capture → camera_ingest validates size, MIME type and JPEG decode → multipart POST /api/camera/occupancy → configured ROI detection → confidence and consecutive-frame checks → persisted accepted occupancy transition → status/dashboard and feature aggregation. Controller flow: RC522 UID → GET /api/parking/status → POST /api/vehicle/entry or /api/vehicle/exit with a persisted idempotency key → response and subsequent status validation → OLED/LED update and one gate cycle. A timeout preserves the pending key and leaves the gate closed until confirmation.

Forecast flow uses the same PreparedDataset contract for each model. The runtime predictor loads compatible, hash-checked model artifacts and uses a deterministic fallback when unavailable. Allocation uses configured weights and excludes occupied candidates. The small five-slot exhaustive benchmark computes the exact single-slot minimum for each scenario.

The backend exposes separate routes for vehicle entry/exit, parking status, image occupancy, allocation benchmarks, forecasts, and dashboard data. The database is the authoritative state shared by those services. Camera-derived observations update occupancy only after the detector's configured confidence and temporal conditions are met; they do not create or close an RFID vehicle session. Conversely, an RFID entry creates an operational session and allocation record but does not prove that a camera has visually observed the vehicle.

5. Hardware Design

The selected compile targets are a generic ESP32 DevKit/WROOM-32 controller and an AI-Thinker ESP32-CAM/OV2640. The repository's original design did not specify purchased board revisions; therefore the controller pin assignment is proposed and the camera firmware explicitly selects the AI-Thinker map. Check the board silkscreen and vendor documentation before power-up. The diagrams are a design proposal, not a physically verified circuit.

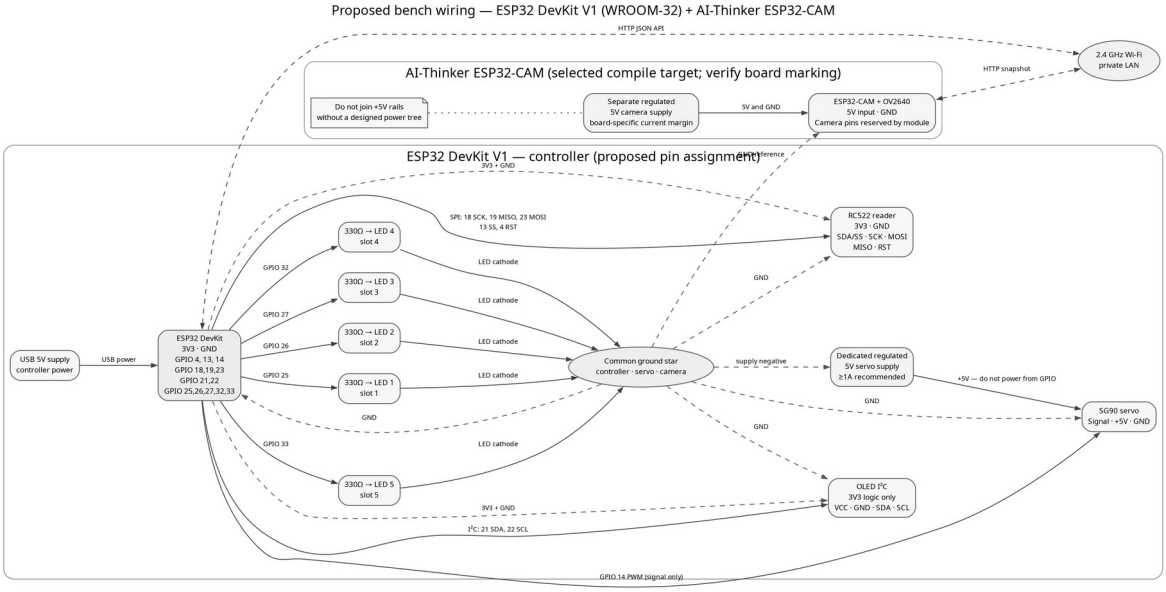


Figure 2. Proposed electrical wiring with separate servo/camera power rails and a common ground reference.

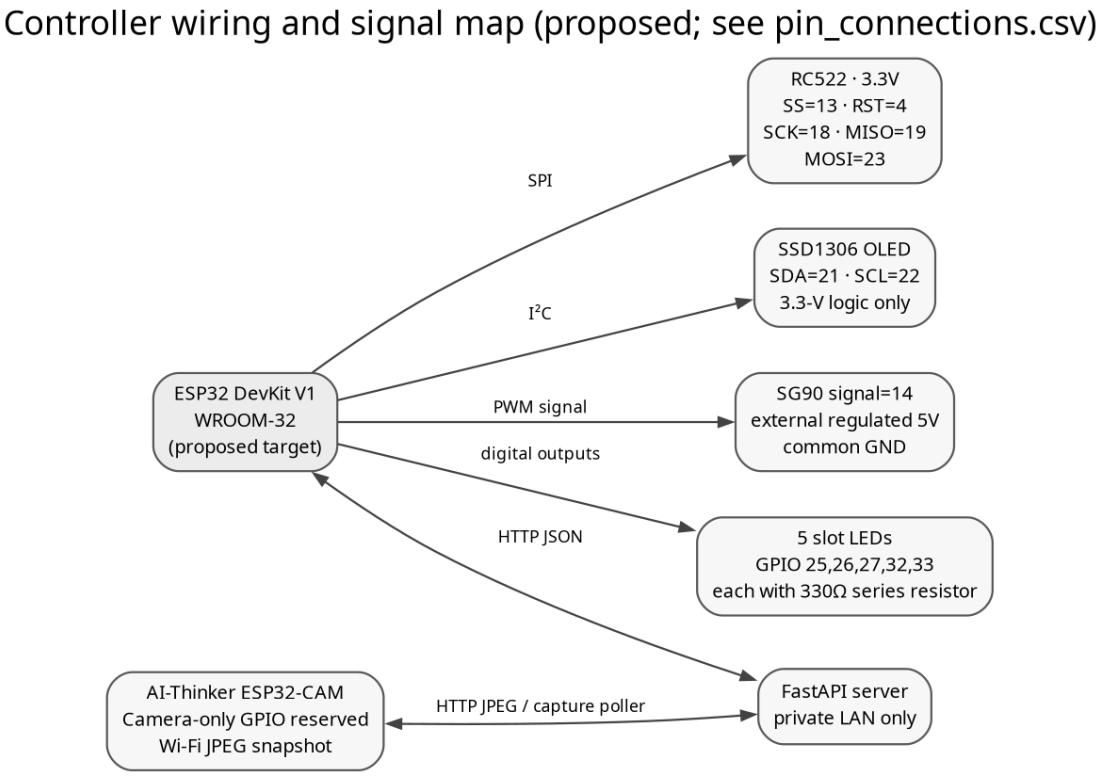


Figure 3. Proposed signal map for controller peripherals and Wi-Fi-connected camera.

Table 3. Proposed controller pin assignments

Component	Signal	ESP32 pin	Voltage	Connection note
RC522	SCK / MISO / MOSI	18 / 19 / 23	3.3 V logic	VSPI; dedicated bus
RC522	SS / RST	13 / 4	3.3 V logic	Avoid strap pins for SS
OLED	SDA / SCL	21 / 22	3.3 V logic	Use compatible pull-ups; I²C address 0x3C
Slot LEDs 1–5	Anodes via 330 Ω	25, 26, 27, 32, 33	3.3 V outputs	Each LED has its own resistor; cathodes to GND
SG90	PWM signal	14	3.3 V signal	Servo +5 V from a separate regulated supply; common GND
AI-Thinker ESP32-CAM	OV2640 camera bus	0, 5, 18, 19, 21–27, 32, 34–39	Module-internal	Reserved by camera; no external peripherals on these pins

The controller pin proposal avoids classic ESP32 strapping pins (0, 2, 5, 12, 15), flash-reserved pins (6–11), and input-only GPIO34–39. The selected RC522 bus uses 18/19/23; the OLED uses 21/22; LED and servo signals use output-capable GPIO. Espressif documents these GPIO restrictions and boot-strap considerations [9]. Camera GPIO assignments follow the AI-Thinker reference map in Espressif's esp32-camera project [10]. The pin map has been reviewed against those references but not checked with a multimeter or actual board revision.

Use the controller's USB input, RC522/OLED 3V3, a separate regulated 5-V supply for the servo, and a separately regulated 5-V camera input. Connect grounds at a common reference while keeping positive 5-V rails isolated. A 330 Ω series resistor limits each indicator LED current. A servo can draw a transient/stall current beyond a GPIO or dev-board regulator budget; size the external supply to the exact SG90 variant, and keep the barrier model lightweight and supervised. A hobby servo is not a safety-rated real gate: loss of servo power may leave the arm where it stopped.

The bill of materials, full per-signal CSV pin map, wiring sequence, calibration notes and troubleshooting table are included in final_submission/hardware/ and docs/hardware/validation.md. The user reports successful operation of the assembled tabletop rig; quantitative per-component validation remains uncaptured.

6. Software and Machine-Learning Design

Table 4. Software components and responsibilities

Technology	Role in implementation
Python 3.14.8	Backend, data preparation, benchmarking, camera ingestion, calibration, report tooling on the measured host.
FastAPI and SQLite	HTTP contracts, lifecycle state, occupancy transitions, prediction/optimizer metrics and dashboard data.
NumPy and Pillow	Feature arrays and bounded JPEG decode/ROI edge-density detection.
TensorFlow/Keras	Server-side LSTM, GRU, and temporal Transformer regression models.
Arduino framework + PlatformIO	ESP32 controller and AI-Thinker ESP32-CAM builds; no device was attached during compile verification.
Graphviz and python-docx	Editable architecture diagrams and generated single-column DOCX/PDF submission.

6.1 Dataset and Preprocessing

The deterministic corpus is explicitly synthetic: 4,032 five-minute timeline buckets across 14 days, 27 model input features, and 6,274 privacy-safe source observations. It covers five configured slots from 2026-08-01T00:00:00Z through 2026-08-14T23:55:00Z; seed 20260908. Features include occupancy ratio/change and rolling means, cyclic hour and weekday terms, simulated weather categories, holiday/event flags, entry/exit/allocation counts, active sessions, privacy-safe vehicle aggregates, and normalized allocation context. The target is occupancy ratio at the next five-minute interval.

The existing preprocessor creates a 12-bucket (60-minute) lookback and one-step forecast horizon. It divides the timeline chronologically into 70% training, 15% validation and 15% test before sequence construction; sequence counts are 2,810, 592 and 594. Continuous-feature means and scales are fitted on training data only. The same immutable PreparedDataset is passed to all three neural models, and no RFID UID is part of model inputs. This prevents future observations from entering earlier sequences or fitted scaling statistics.

Missing-value handling follows the existing feature builder: eligible short gaps are forward-filled and longer gaps remain missing and are excluded from training examples. Bounded/cyclic/one-hot inputs retain their established contract; the continuous feature scaler is shared by all models. The last timeline bucket has no next-step target and is not used as a supervised target row. The data is useful for exercising code paths and comparing models reproducibly; it cannot estimate field performance, actual weather effects, arrival rates, camera accuracy or transfer to another lot.

Dataset files and metadata are in review_outputs/2026-10-15/dataset. A public, version-pinned ZIP download is available from GitHub: download simulated dataset

6.2 Forecasting Architectures

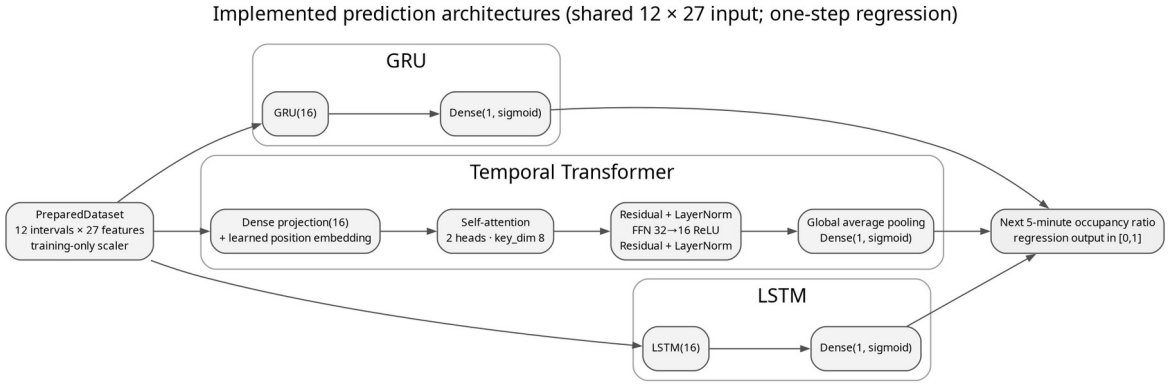


Figure 4. Implemented neural architectures; each receives the same scaled 12×27 sequence and predicts a bounded scalar.

LSTM and GRU models each apply one 16-unit recurrent layer to the 12-step sequence, followed by a one-unit sigmoid output. The temporal Transformer projects each 27-feature step to width 16, adds learned positional embeddings, applies two-head self-attention (key dimension 8), residual/LayerNorm, a 32-unit ReLU then 16-unit feed-forward block, a second residual/LayerNorm, global average pooling, and a sigmoid scalar output. Training uses mean-squared-error loss with Adam at 0.001, 12 maximum epochs, batch size 32, early-stopping patience 3, recurrent units 16, Transformer width 16 and two heads. Seeded TensorFlow/Keras deterministic operations are enabled where supported. The target is continuous occupancy ratio, so classification accuracy, F1, ROC and confusion matrices are not applicable.

Model files use content-addressed Keras artifacts with manifests containing feature order, split identity, scaler values, configuration, software versions and SHA-256. The artifacts execute in the server runtime. TensorFlow Lite, quantization and pruning were not implemented because inference is intentionally server-side; no ESP32 inference claim is made.

6.3 Allocation Objective

For candidate slot i, the shared normalized scalar objective is $J(i) = 0.35\ d_i + 0.30\ c_i + 0.20\ w_i + 0.15\ f_i$, where distance d, congestion c, waiting contribution w, and fairness penalty f are read from parking configuration. Occupied slots are excluded before search. The benchmark compares a first-available policy, exhaustive enumeration of every free slot (the exact optimum for this single-slot decision), and GA, PSO, and ACO on identical scenario records and weights. For heuristic assignment x, the optimality gap is $J(x) - \min_i J(i)$, which is nonnegative up to floating-point tolerance. Convergence curves are not included because the existing optimizer interface does not expose valid per-iteration histories [12], [13].

The controller-side request sequence is:

```
read and normalize UID in RAM
if a saved operation exists: retry its same idempotency key
else: inspect current server status; choose entry or exit
persist operation + key in NVS before HTTP POST
validate response identity and gate_action; re-read status
if committed status matches: persist gate_done before one gate cycle
on timeout/conflict/malformed reply: keep gate closed
```

7. Implementation Details

The backend lifecycle API accepts JSON fields rfid_uid and idempotency_key. A key is generated from the controller chip identifier and an NVS-persisted sequence number, then stored with the pending operation before transmission. A restart reuses the same key and payload, allowing the server's existing idempotency behavior to replay a completed response safely. The UID is not printed to Serial. As the existing API has no device authentication or TLS provision in this project, these requests must remain on a trusted private LAN.

Allocation follows one shared weighted_cost function. A no-available-slot response is HTTP 409, which the controller treats as a closed-gate result. For successful entry, the firmware validates the returned UID, slot

identifier, slot name, gate_action and current status before display and actuation. For exit it requires that the session no longer appears active. The slot LEDs reflect accepted server occupancy: ON means occupied, OFF means free, and all blink when status is unknown. OLED messages cover initialization, Wi-Fi/backend state, scanning, assignment, parking-full/conflict, completion and errors. The timed gate state machine closes after its configured open interval; it does not use an unbounded delay.

The ESP32-CAM exposes GET /health and GET /capture. AI-Thinker pin assignments are isolated in camera_board.h; unknown board variants fail compilation unless a mapping is explicitly selected. JPEG capture uses VGA and JPEG quality 12 by default, with PSRAM-aware buffers. The Python camera_ingest process validates image/jpeg, bounds payload size and response size, decodes the JPEG with Pillow, posts to the existing multipart occupancy API, limits operation to one in-flight frame, and exponentially backs off after errors. The minimum capture interval is two seconds.

ROI calibration captures a live JPEG or accepts a previously saved real JPEG, presents an interactive Tk/Pillow rectangle selector, validates one in-bounds non-overlapping ROI per configured slot, writes the same ROI schema used by the detector, and saves a visual overlay. This environment cannot supply a real lot image, so no calibrated field ROI file is included. The current detector computes edge density for each crop and applies the configured score threshold; confidence and three consecutive candidate frames are required before persisting a transition. It remains a heuristic, not a validated vehicle classifier.

8. Experimental Setup

The prediction experiment uses the generated synthetic timeline and the unchanged chronological split. All neural models use common preprocessing and the untouched test partition; the persistence baseline predicts the latest occupancy ratio. Inference latency is measured as median warm prediction time divided by example count on the server CPU. The results are from TensorFlow 2.22.0-rc0/Keras 3.16.0.dev2026101404 under Python 3.14.8 on Linux x86_64 with an Intel Core i7-14650HX host. No accelerator or microcontroller timing is reported.

Optimizer tests use 120 distinct scenarios, three repeat seeds per scenario, 30 iterations per heuristic, and configured weights {"congestion": 0.3, "distance": 0.35, "fairness": 0.15, "waiting": 0.2}. Scenario occupancy spans empty, light, moderate, heavy, near-full and full categories. Seed derivation is `20261015 + scenario_index * 101 + repetition`. Each policy receives identical scenario inputs. Runtime is measured with a monotonic clock. Full-capacity rejections are separated from successful cost statistics. This is a simulated one-slot objective, not traffic throughput or physical wait time.

The host's development environment provides an Intel Core i7-14650HX with 24 CPUs online and 15 GiB RAM; benchmark latencies are software measurements from this machine and vary with system load. During an earlier integration check, the host exposed a generic CH340 USB serial adapter at /dev/ttyUSB0, but the attached target board could not then be identified from the host. No private controller configuration was present, and esp32-cam.local was not resolved at that time. Later work identified and flashed an ESP32 controller; the user subsequently reported that the assembled tabletop rig works. These historical host observations are not post-rewire peripheral measurements.

9. Results and Discussion

9.1 Prediction Results

Table 5 compares the persistence reference with the three trained neural regressors on the same test windows. GRU is the best measured model for MAE and RMSE on this generated split. This is an empirical ranking for one synthetic dataset and seed, not evidence of universal superiority or real-lot forecast quality. All inference timing is server CPU timing; model size is serialized Keras size.

Table 5. Test-set prediction performance by model

Model	MAE	MSE	RMSE	Latency ms/example	Size bytes
persistence_baseline	0.120875	0.028620	0.169173	0.000015	N/A
lstm	0.104062	0.016818	0.129685	0.245910	60481
gru	0.098055	0.013891	0.117862	0.223783	52615

Model	MAE	MSE	RMSE	Latency ms/example	Size bytes
transformer	0.109173	0.018098	0.134531	0.232451	110684

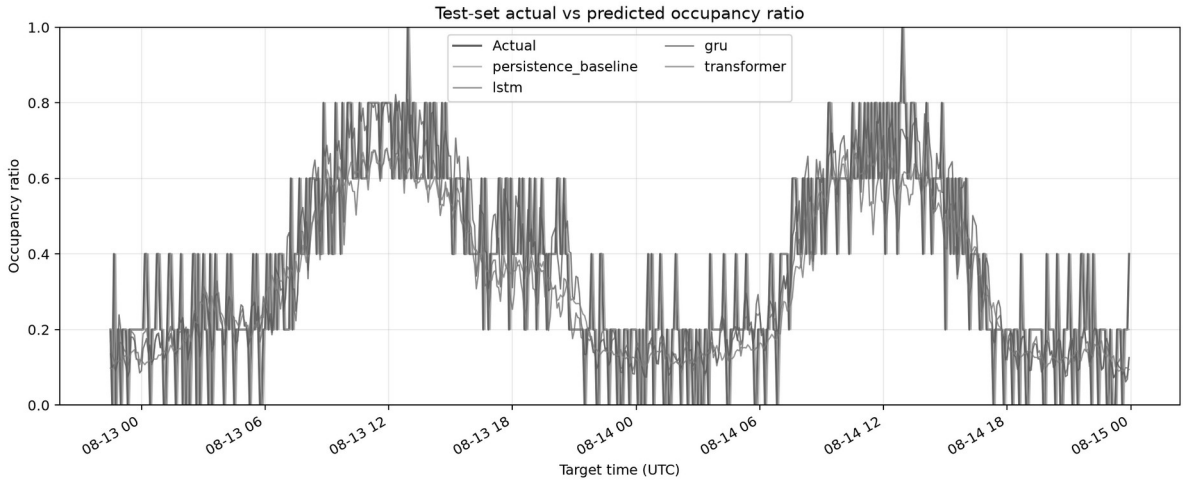


Figure 5. Test-window actual versus predicted occupancy ratio from the executed model comparison.

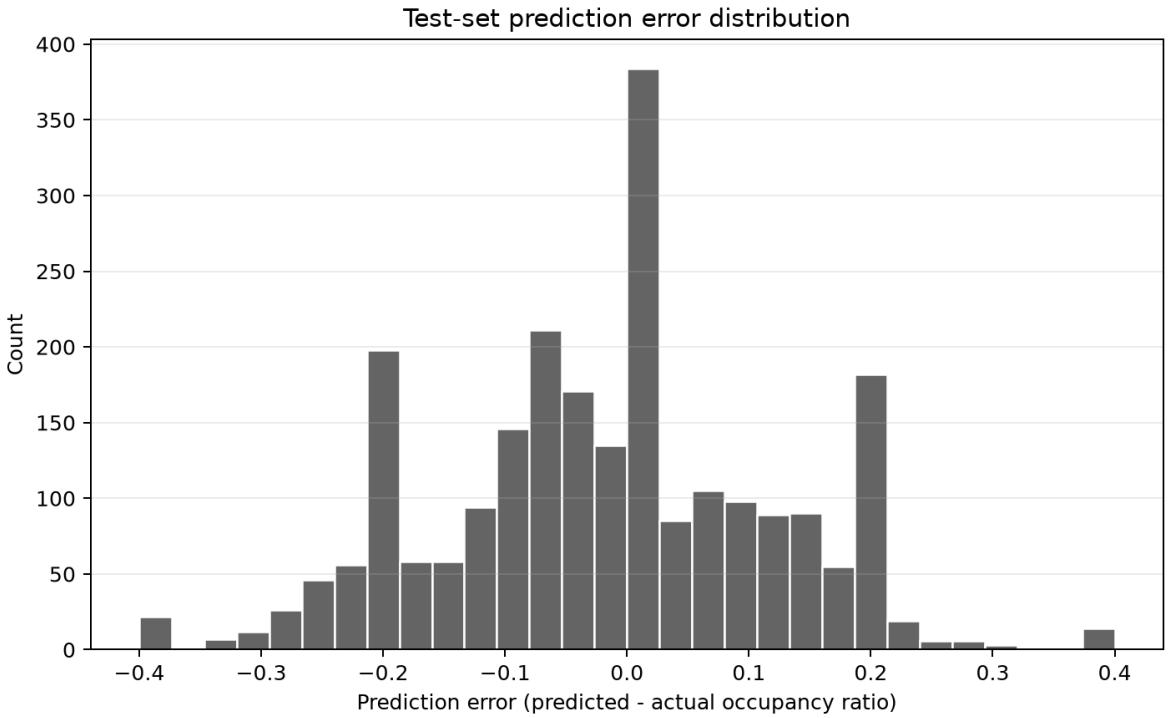


Figure 6. Prediction error distribution; errors are regression residuals, not class labels.

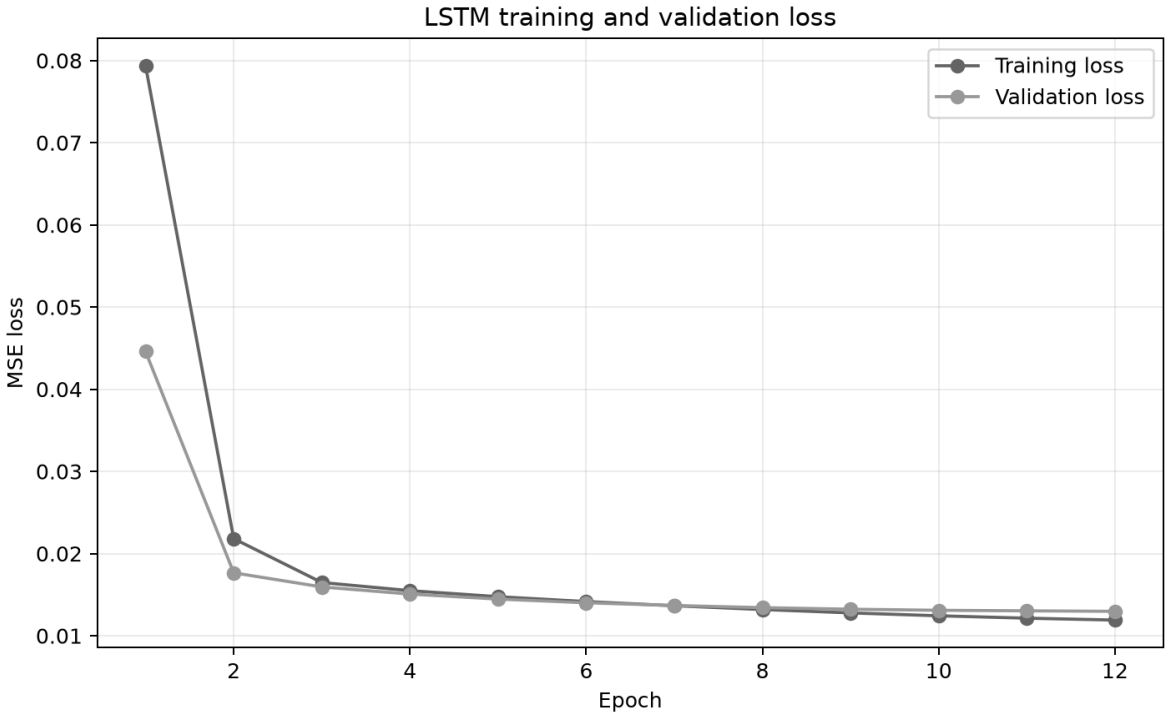


Figure 7. LSTM training and validation loss for the fitted run.

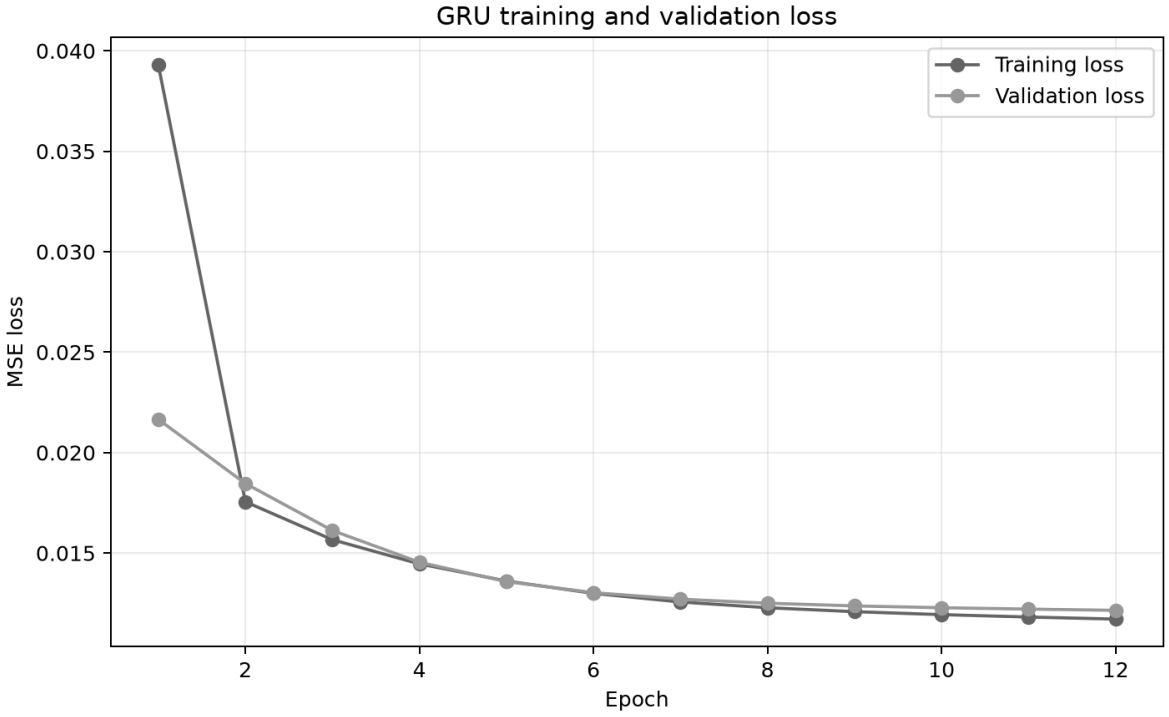


Figure 8. GRU training and validation loss for the fitted run.

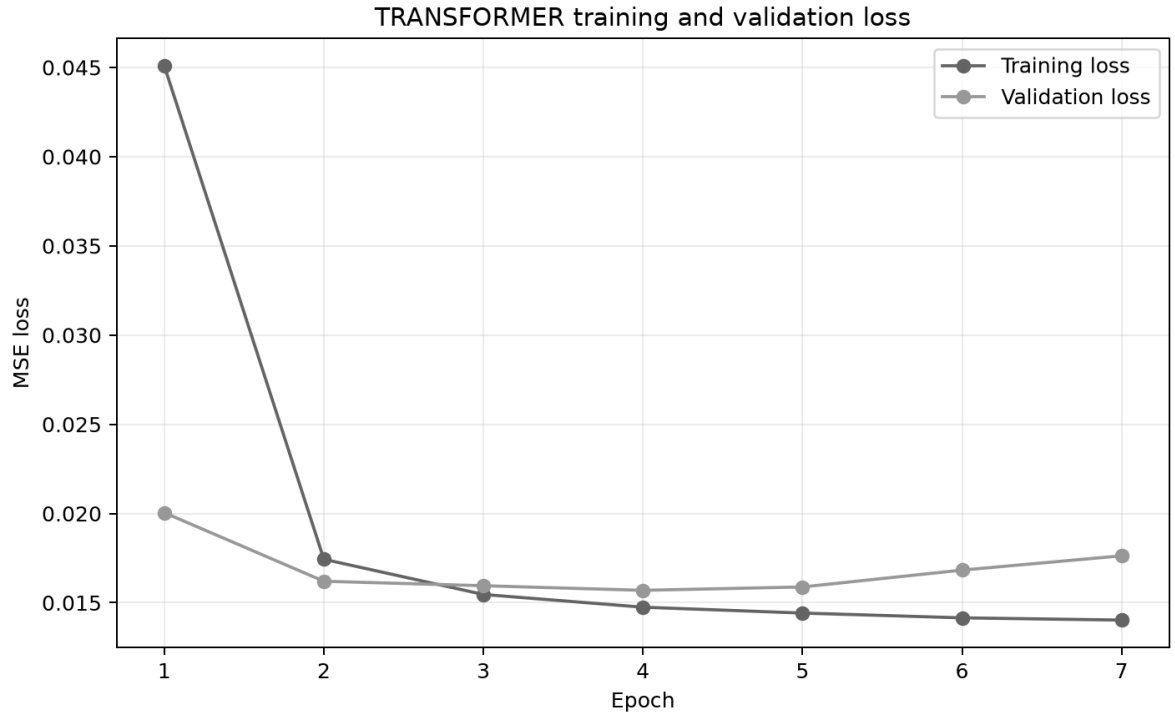


Figure 9. Transformer training and validation loss for the fitted run.

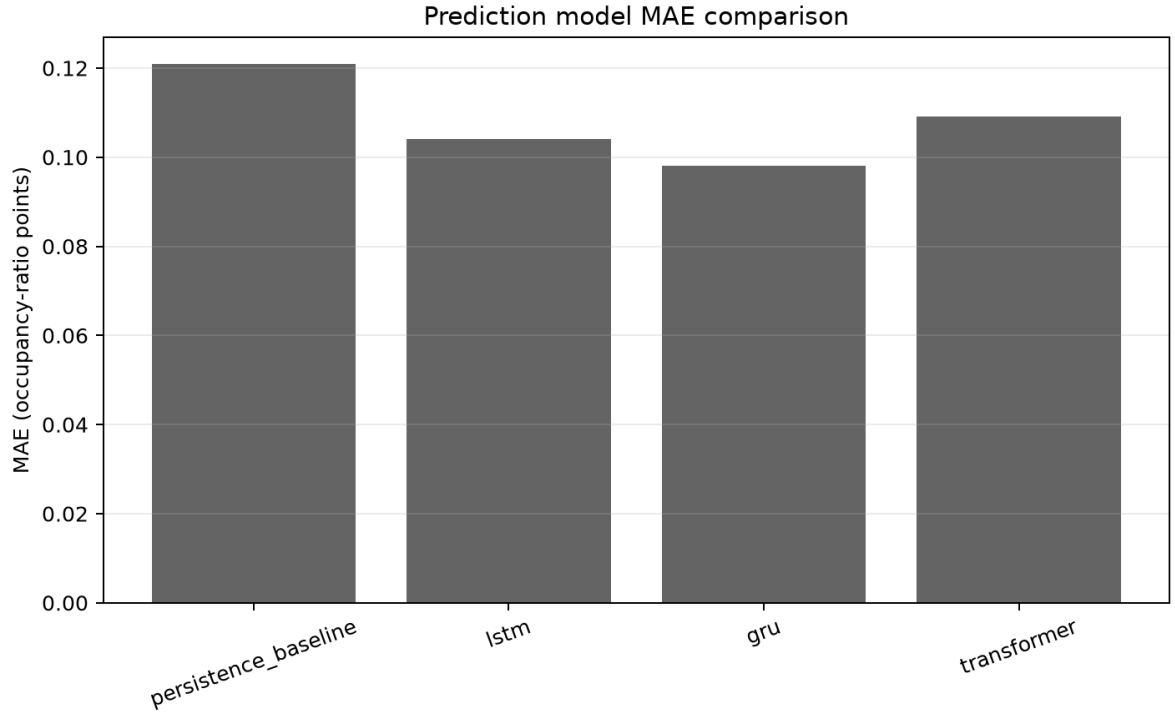


Figure 10. Measured model MAE comparison on the shared chronological test partition.

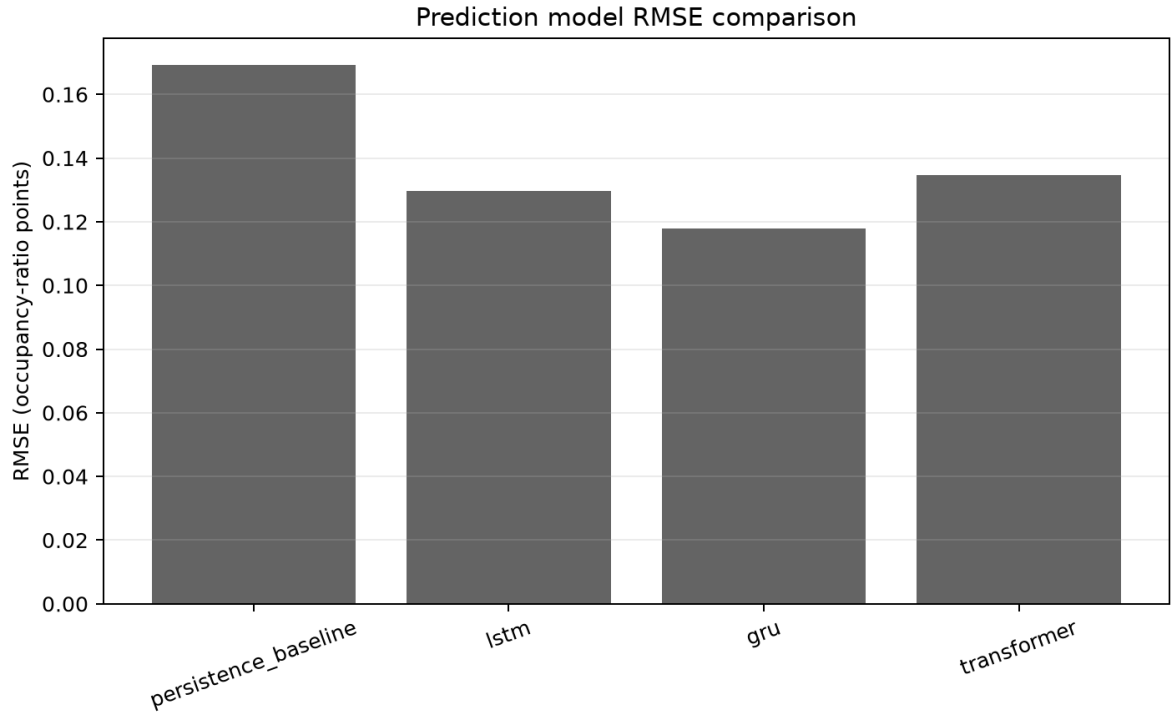


Figure 11. Measured model RMSE comparison on the shared chronological test partition.

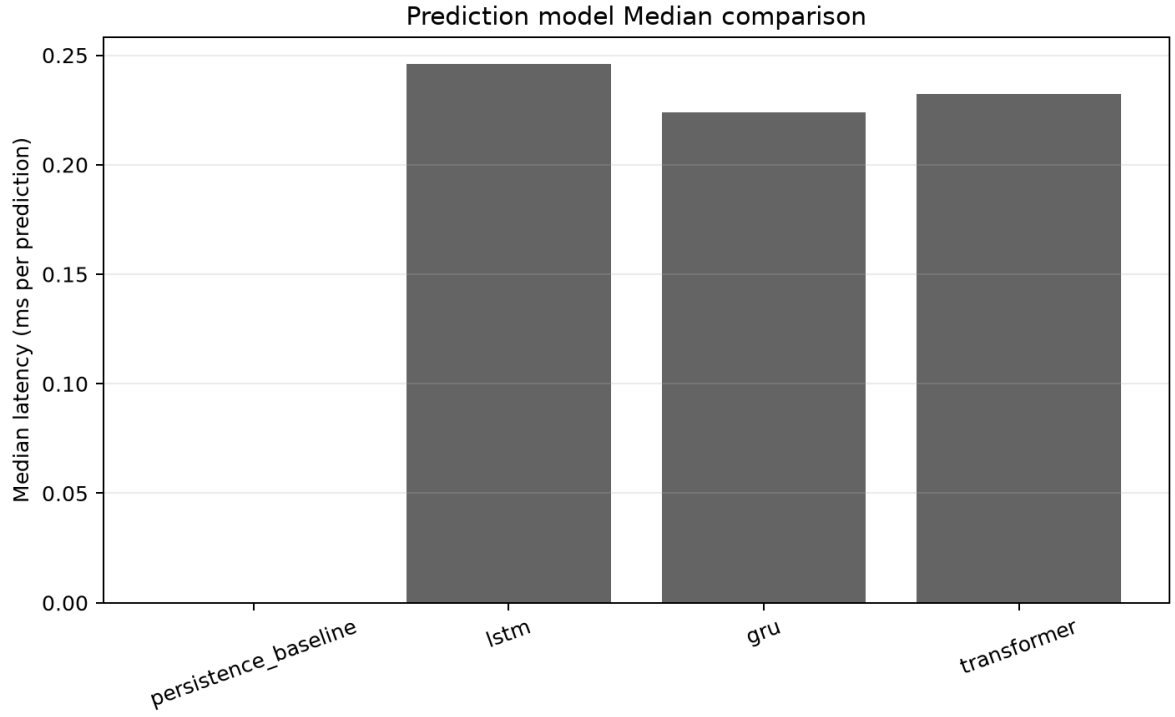


Figure 12. Measured per-example inference latency on the server host.

The measured GRU test MAE is 0.098055, versus persistence MAE 0.120875. The 27 features include simulated calendar/weather/event context, but synthetic context may encode generator-specific patterns. Real measured samples, external-site validation and repeated independent dataset seeds are needed before drawing deployment conclusions. No camera accuracy metric is reported because no labeled image evaluation set is available.

9.2 Allocation Results

Table 6 summarizes allocation performance across the repeated attempts; each policy rejected 60 of 360 attempts at full capacity. There were zero invalid slot assignments. The exhaustive policy is the exact single-slot minimum

for each scenario. GA matched this mean cost in this run but required substantially more Python runtime than enumeration; ACO was close to optimal and PSO had a larger mean gap. The first-available baseline was fastest but had higher mean cost. Heuristics are not expected to beat exact enumeration when at most five candidates exist.

Table 6. Allocation performance across all repeated runs

Policy	Success / attempts	Mean cost	Median runtime ms	Mean gap	Invalid
first_available	300 / 360	0.487125	0.000331	0.108931	0
exhaustive_optimum	300 / 360	0.378194	0.014984	0.000000	0
GA	300 / 360	0.378194	0.807225	0.000000	0
PSO	300 / 360	0.426458	1.157470	0.048264	0
ACO	300 / 360	0.381779	1.162547	0.003586	0

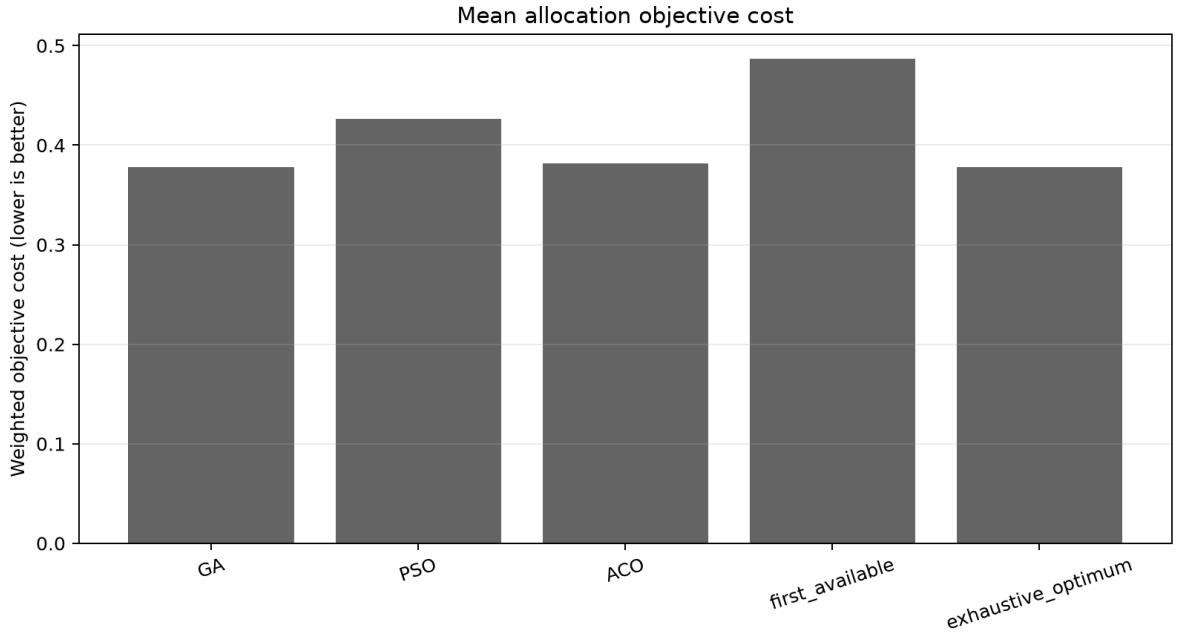


Figure 13. Mean allocation objective cost by policy and exact/baseline reference.

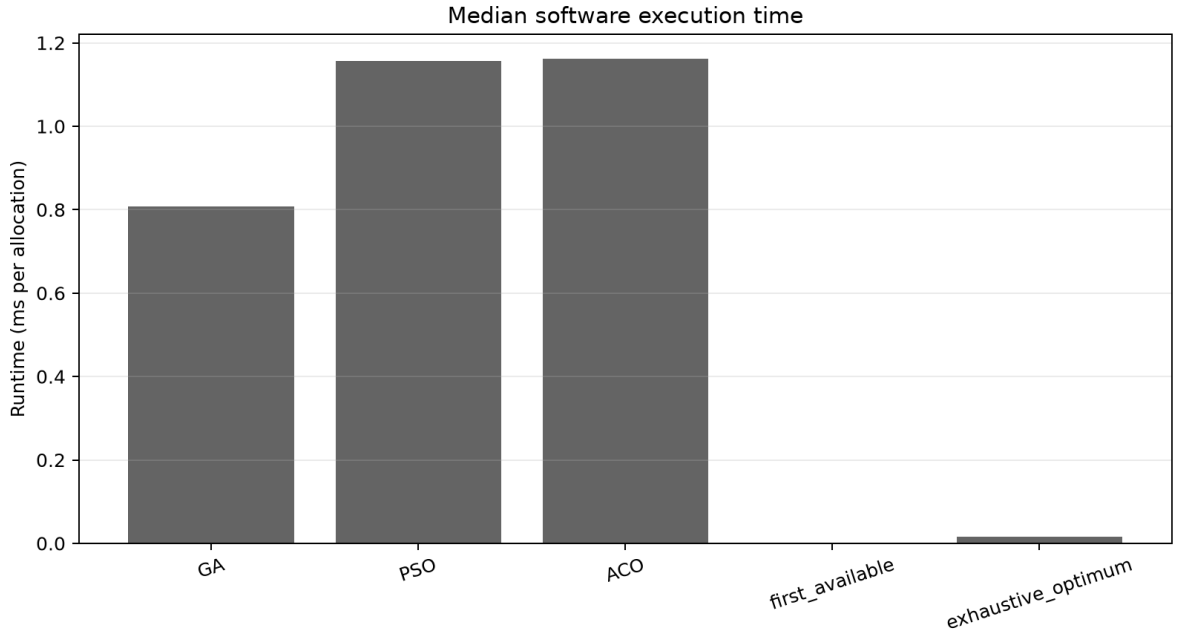


Figure 14. Software execution-time comparison; logarithmic scaling is labeled in the figure.

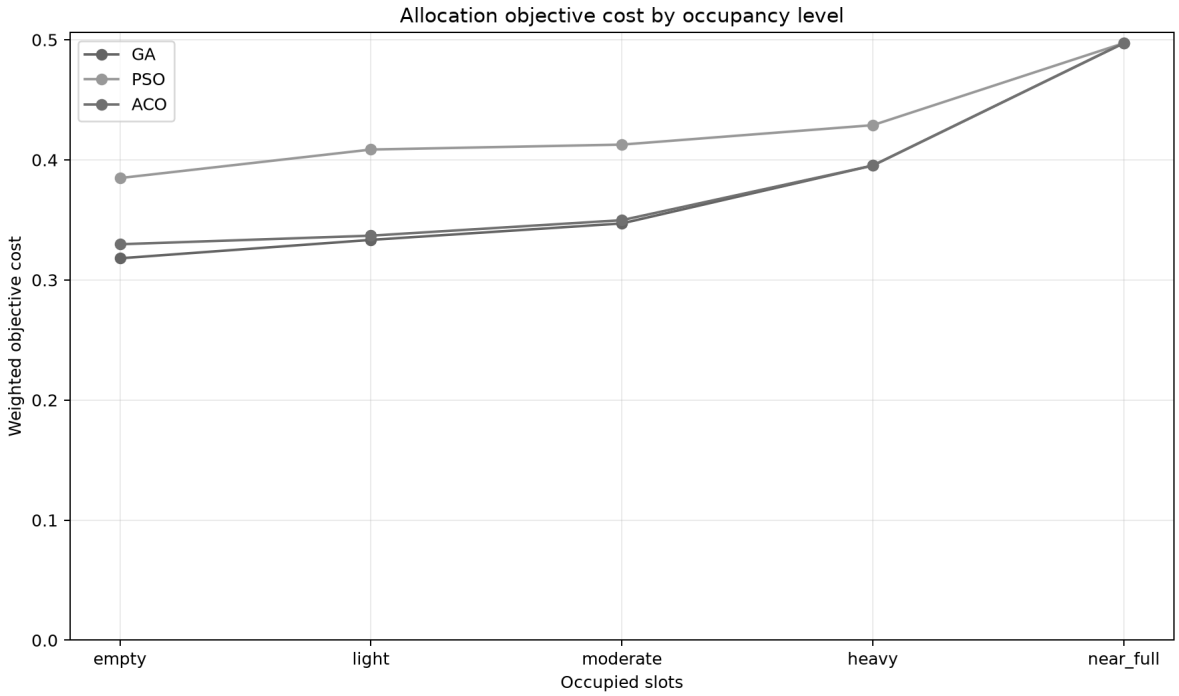


Figure 15. Objective cost by simulated occupancy category.

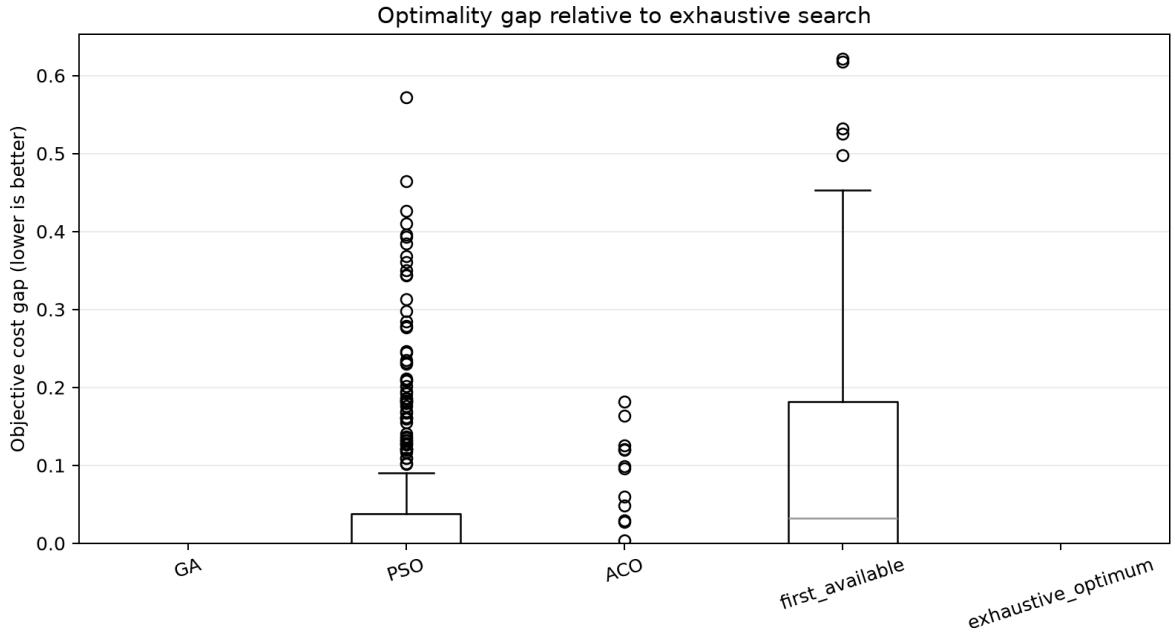


Figure 16. Per-run nonnegative gap from exact enumeration.

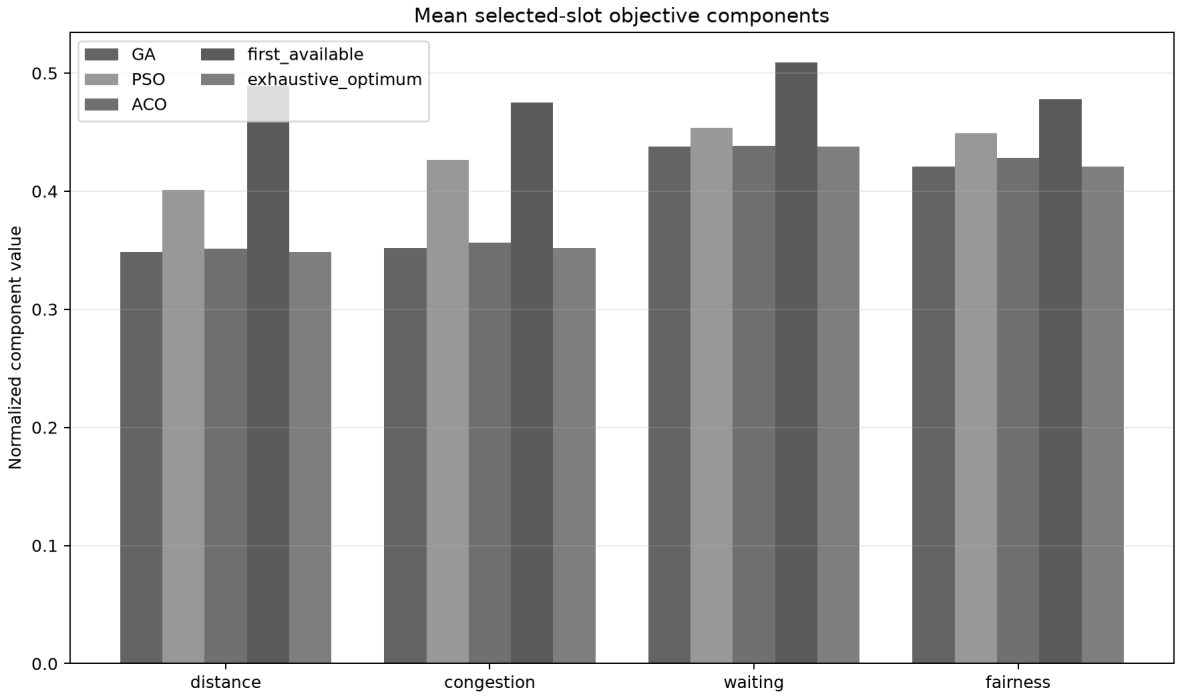


Figure 17. Mean weighted objective components among successful allocations.

Table 7. Mean objective cost by simulated occupancy category

Occupancy category	First available	Exact	GA	PSO	ACO
empty	0.508157	0.318020	0.318020	0.384866	0.329726
light	0.481549	0.333373	0.333373	0.408601	0.336888
moderate	0.446407	0.347081	0.347081	0.412717	0.349788
heavy	0.502314	0.395297	0.395297	0.428909	0.395297
near_full	0.497197	0.497197	0.497197	0.497197	0.497197

The occupancy-stratified table uses 60 repeated successful runs per category and policy. Full-capacity cases are excluded from these cost means and are reported separately. Runtime values describe Python algorithm execution on the host and are not estimates of physical vehicle wait or lane throughput.

9.3 Hardware and integration test status

PlatformIO compiled both firmware targets. A read-only probe identified the controller as ESP32-D0WD-V3 revision 3.1; its 4,194,304-byte flash was backed up before the upload, and esptool verified the flashed image hash. Serial telemetry recorded 3 boots and 3 Wi-Fi connections. The local API access log recorded 5 successful health checks and 5 authenticated parking-status responses from the ESP32 after its latest boot. Free heap across 8 physical serial events averaged 274069 bytes (median 292240, sample standard deviation 26019, p95 293416). The RC522 VersionReg probes returned 0xFF, 0x00 during the earlier wiring attempt. The user subsequently reports that the rewired tabletop rig works, but no post-rewire serial capture, trial count, or quantitative result was supplied; this remains a user-reported functional check rather than independently captured evidence. No camera health endpoint was found during the earlier LAN scan. RFID success rate, camera accuracy, OLED/LED correctness, current draw, and physical servo timing remain without measured trials. The occupancy API has software tests with generated images and status-transition assertions, not a labeled real-camera set.

Table 8. Verification evidence and physical test status

Check	Observed result
Controller firmware build	PASS: compiled and flashed to ESP32-D0WD-V3 revision 3.1; esptool hash verification passed and a full flash readback was saved before update.

Check	Observed result
ESP32-CAM firmware build	PASS: PlatformIO compilation for AI-Thinker target; no camera capture verified.
USB serial/network reachability	PASS: 3 Wi-Fi connection events; 5 authenticated status responses returned HTTP 200.
RC522 reader and card scan	Earlier probe: invalid VersionReg values 0xFF, 0x00. The user later reported successful operation after rewiring; no post-rewire card-read log or repeated-trial count was captured. Qualitative user report only.
OLED, LEDs, servo, physical camera	The user reports that the assembled rig works; no post-rewire per-component captures or trial counts are available. NOT TESTED: camera board or /health endpoint was not found on LAN.
Vision accuracy	Not measured; no labeled real-camera test set.
ESP32 inference latency / memory	Not applicable to the current server-side TensorFlow design; not measured on device.

9.4 Source- and specification-based engineering estimates

The following are rough engineering planning estimates, not hardware measurements or simulated benchmark outputs. Rate and latency ranges are low-confidence analyst assumptions for a close-range tabletop setup on stable local Wi-Fi. Component timing and current ranges use firmware constants and manufacturer specifications where available. Replace these with recorded trials before drawing performance conclusions. Camera accuracy has no defensible estimate without labeled images.

Table 9. Rough engineering estimates (not measured)

Quantity	Estimate	Basis and limitations
RFID successful reads per close, aligned tap	90–99% (very low confidence)	Assumes a supported tag within about 20 mm, stable 3.3 V, and no nearby metal. MFRC522 specifies up to 50 mm depending on antenna tuning; that is not a success-rate guarantee [20].
API entry/exit transaction success on stable local LAN	98–99.5% (very low confidence)	Planning assumption for a reachable backend and healthy Wi-Fi. Retries are idempotent, but no post-rewire transaction trial series was captured.
Camera capture success on stable power/Wi-Fi	95–99% (very low confidence)	Planning assumption only; no physical ESP32-CAM capture series or dropped-frame count is available.
LED correctness with known slot state	95–100% (very low confidence)	Assumes correct pin mapping/polarity and a successful status refresh. No per-slot expected-versus-observed trial table is available.
Entry or exit latency, tag detection to accepted API response	0.2–1.0 s (low confidence)	Order-of-magnitude estimate for one local-Wi-Fi request and SQLite transaction with the server healthy. The 3.5 s HTTP timeout is a failure limit, not expected latency.
Wi-Fi recovery after AP becomes available	5–15 s (low confidence)	Firmware retries every 5 s; range allows up to about 10 s for association/DHCP after a retry. Poor AP conditions may take longer or fail.
Servo command delay, RFID detection to authorized GPIO command	0.2–1.0 s (low confidence)	Uses the assumed local API round trip; firmware issues the command only after authorization. Excludes mechanical travel.
SG90 travel for configured 10° → 85° sweep	0.13–0.50 s (very low confidence)	Ideal nominal is 75° × 0.1 s/60° ≈125 ms at 4.8 V; upper range is a load/settling allowance. Exact servo variant is unverified [19].
OV2640 frame capture latency, configured JPEG mode	0.10–0.50 s (very low confidence)	Planning range for one SVGA-class JPEG frame with PSRAM. Exposure, frame size, clone, and lighting affect timing; no physical capture log is available.

Quantity	Estimate	Basis and limitations
Backend five-ROI heuristic processing latency	0.01–0.10 s (very low confidence)	Planning range for one frame and five ROIs on the local host; excludes capture, transfer, and queue delay. Not measured on a physical frame.
Controller free heap	Measured historical mean 274,069 bytes (n=8)	Captured physical serial telemetry before the user's later hardware report; median 292,240 bytes, SD 26,019 bytes, p95 293,416 bytes. This is measured, not an estimate or future guarantee.
Controller + reader + OLED + LEDs input current, servo excluded	0.15–0.40 A at 5 V (very low confidence)	Budget estimate from ESP32 active Wi-Fi radio draw plus peripherals; regulators, OLED brightness, LED current, and network duty cycle vary [21].
AI-Thinker ESP32-CAM input current	About 0.18 A at 5 V; up to 0.31 A with flash at maximum	Ai-Thinker module specification; not measured on this camera board [22].
SG90 servo supply current	0.5–2.0 A (very low confidence)	Manufacturer product page notes this operating-current range; exact unit/load is unknown. Use a separate regulated servo supply, never ESP32 GPIO or 3.3 V [19].
Camera occupancy accuracy	Not estimated	Requires a manually labeled real-scene test set. Synthetic frames cannot support an accuracy percentage.

10. Testing and Demonstration

Automated software coverage includes backend lifecycle retries/conflicts, no-free-slot behavior, JPEG input validation, ROI bounds, confidence smoothing, state persistence, prediction fallback/artifact compatibility, simulated demo API ordering, and the new camera-ingestion and ROI-geometry tests. The build checklist distinguishes those results from firmware compilation and physical tests. The hardware checklist records the controller boot, Wi-Fi, and authenticated status API evidence separately from the earlier invalid RC522 SPI probes. The user later reported successful operation after rewiring, but no post-rewire per-component logs, trial counts, or power measurements were captured.

A separate host-runtime smoke test copied the review database, selected the saved GRU artifact through the existing predictor setting, and called GET /api/prediction. The response was model=gru, fallback_reason=null, occupancy_ratio=0.136493 for the synthetic timeline. This verifies server-side artifact loading and inference; it is not an ESP32 measurement.

The same-host simulated integration check also confirmed /health with five configured slots, accepted three generated JPEG frames through the occupancy route, and ran a labeled simulated RFID entry that received slot S1. GA, PSO and ACO benchmark endpoints returned results. With only the short synthetic history in that isolated database, prediction correctly reported the deterministic fallback reason insufficient_history. These checks establish that software paths respond; generated images and RFID values do not establish camera or reader operation.

For a simulated review: activate the project's virtual environment and run `make test` and `make lint`; start the API with `make run`; execute `python -m scripts.run_simulated_demo --base-url http://127.0.0.1:8000 --rfid-uid 04A1B2C3 --idempotency-key final-review-entry-001`; open http://127.0.0.1:8000/dashboard; then show `review_outputs/2026-10-15/EXPERIMENTAL_REPORT.md` and the figures under `review_outputs/2026-10-15/final_submission/figures/`. The simulated JPEGs and RFID label must be described as simulated. Do not run `make review` during the presentation because it retrains models and regenerates result files.

For a physical demo after wiring validation: use the private-LAN addresses in local_config.h, confirm GET /health, GET http://CAMERA-IP/health, run camera_ingest with a five-second interval, and scan one test tag. Verify status and gate action before proceeding to subsequent cards. Keep a hand-operated emergency power disconnect available for the lightweight servo fixture. The detailed startup, cleanup, and failure procedure appears in DEMO_INSTRUCTIONS.md.

11. Limitations and Future Work

Primary limitations are the synthetic dataset, five-space scale, one-step horizon, generator-specific correlations, edge-density camera heuristic, absent labeled real imagery, lack of post-rewire quantitative hardware logs, and nonauthenticated HTTP control API. The user reports that the assembled tabletop rig works, but the available serial evidence predates that report. The architecture assumes a fixed camera and calibrated static ROIs. Lighting, weather, vehicle type, partial occlusion, perspective, shadows and camera motion may invalidate the baseline detector. Prediction accuracy on synthetic samples cannot establish field generalization.

Next work should acquire permissioned, labeled images and real occupancy timelines; split evaluation by day/site and repeat seeds; compare the heuristic with a measured vision baseline; validate the AI-Thinker and controller board revisions; conduct current-limited bench tests for RFID, display, LEDs and a lightweight servo; add transport authentication for device requests before any wider network exposure; and measure field latency and availability. A real gate requires a safety-rated actuator/controller and independent interlocks; the SG90 prototype is not that system.

12. Conclusion

The repository provides a reproducible software smart-parking prototype with lifecycle, occupancy, allocation, forecasting, dashboard, synthetic experiment outputs, and ESP32 controller/camera firmware plus camera ingestion and ROI calibration. The measured synthetic experiment ranks GRU lowest in test MAE among the tested predictors, while exhaustive enumeration is an efficient exact reference for five-slot allocation. These results demonstrate functioning software and an experimental workflow, not deployment performance. The user reports successful operation of the assembled tabletop hardware. Earlier serial probes predate that report, and quantitative post-rewire RFID, camera, display, LED, servo, power, and latency trials were not captured. The report separates this user-reported function from measured results and source/specification-based estimates.

References

[1] T. Lin, H. Rivano, and F. Le Mouël, “A Survey of Smart Parking Solutions,” IEEE Transactions on Intelligent Transportation Systems, vol. 18, no. 12, pp. 3229–3253, 2017. doi: 10.1109/TITS.2017.2685143.

[2] E. E. Tsiropoulou, J. S. Baras, S. Papavassiliou, and S. Sinha, “RFID-based smart parking management system,” Cyber-Physical Systems, vol. 3, nos. 1–4, pp. 22–41, 2017. doi: 10.1080/23335777.2017.1358765.

[3] P. R. L. de Almeida, L. S. Oliveira, A. S. Britto Jr., E. J. Silva Jr., and A. L. Koerich, “PKLot – A robust dataset for parking lot classification,” Expert Systems with Applications, vol. 42, no. 11, pp. 4937–4949, 2015. doi: 10.1016/j.eswa.2015.02.009.

[4] S. Yang, W. Ma, X. Pi, and S. Qian, “A deep learning approach to real-time parking occupancy prediction in transportation networks incorporating multiple spatio-temporal data sources,” Transportation Research Part C: Emerging Technologies, vol. 107, pp. 248–265, 2019. doi: 10.1016/j.trc.2019.08.010.

[5] D. Thomas and B. C. Kovoor, “A Genetic Algorithm Approach to Autonomous Smart Vehicle Parking system,” Procedia Computer Science, vol. 125, pp. 68–76, 2018. doi: 10.1016/j.procs.2017.12.011.

[6] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735 –1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[7] K. Cho et al., “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” in Proceedings of EMNLP, 2014, pp. 1724–1734. doi: 10.3115/v1/D14-1179.

[8] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems 30, 2017. arXiv:1706.03762.

[9] Espressif Systems, “GPIO & RTC GPIO — ESP32,” ESP-IDF Programming Guide v5.1. https://docs.espressif.com/projects/esp-idf/en/v5.1/esp32/api-reference/peripherals/gpio.html (accessed 14 Oct. 2026).

[10] Espressif Systems, “ESP32-CAM / AI-Thinker camera pin map,” esp32-camera repository. https://github.com/espressif/esp32-camera/blob/master/examples/camera_example/main/camera_pinout.h (accessed 14 Oct. 2026).

[11] Smart Parking System repository, implementation and experimental outputs. https://github.com/dumbly-smart/smart-parking (accessed 14 Oct. 2026).

[12] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in Proceedings of IEEE International Conference on Neural Networks, vol. 4, 1995, pp. 1942–1948. doi: 10.1109/ICNN.1995.488968.

[13] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant System: Optimization by a Colony of Cooperating Agents,” IEEE Transactions on Systems, Man, and Cybernetics, Part B, vol. 26, no. 1, pp. 29–41, 1996. doi: 10.1109/3477.484436.

[14] P. R. L. de Almeida, J. H. Alves, R. S. Parpinelli, and J. P. Barddal, “A systematic review on computer vision-based parking lot management applied on public datasets,” Expert Systems with Applications, vol. 198, article 116731, 2022. doi: 10.1016/j.eswa.2022.116731.

[15] C. P. Lee, F. T. J. Leng, R. A. A. Habeeb, M. A. Amanullah, and M. H. ur Rehman, “Edge computing-enabled secure and energy-efficient smart parking: A review,” Microprocessors and Microsystems, vol. 93, article 104612, 2022. doi: 10.1016/j.micpro.2022.104612.

[16] Y. Feng, Y. Xu, Q. Hu, S. Krishnamoorthy, and Z. Tang, “Predicting vacant parking space availability zone-wisely: A hybrid deep learning approach,” Complex & Intelligent Systems, vol. 8, pp. 4145–4161, 2022. doi: 10.1007/s40747-022-00700-1.

[17] F. Zhang, K. Shang, L. Yan, H. Nan, and Z. Miao, “Prediction of Parking Space Availability Using Improved MAT-LSTM Network,” ISPRS International Journal of Geo-Information, vol. 13, no. 5, article 151, 2024. doi: 10.3390/ijgi13050151.

[18] F. J. Enríquez, J.-M. Mejía-Muñoz, G. Bravo, and O. Cruz-Mejía, “Smart Parking: Enhancing Urban Mobility with Fog Computing and Machine Learning-Based Parking Occupancy Prediction,” Applied System Innovation, vol. 7, no. 3, article 52, 2024. doi: 10.3390/asi7030052.

[19] TowerPro, “SG90 Digital,” manufacturer specifications, operating speed 0.1 s/60° at 4.8 V. https://towerpro.com.tw/product/sg90-7/ (accessed 21 Oct. 2026).

[20] NXP Semiconductors, “MFRC522 Standard performance MIFARE and NTAG frontend,” Rev. 3.9, 2016. Typical read/write distance up to 50 mm depending on antenna size and tuning. https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf (accessed 21 Oct. 2026).

[21] Espressif Systems, “ESP32 Series Datasheet,” v5.3, Wi-Fi active-mode current. https://documentation.espressif.com/esp32_datasheet_en.html (accessed 21 Oct. 2026).

[22] Shenzhen Ai-Thinker Technology, “ESP32-CAM Wi-Fi+BT SoC Module Product Specification,” v1.0, 2017. https://d229kd5ey79jzj.cloudfront.net/1205/ESP32-CAM%20Product%20Specification.pdf (accessed 21 Oct. 2026).

Dataset Availability

The privacy-safe simulated dataset archive is publicly downloadable from the project's GitHub repository. The version-pinned ZIP contains parking_raw.csv, parking_features.csv, dataset_metadata.json, and dataset documentation. A Google Drive mirror is also provided, but its current permission is restricted to the vit.ac.in Google Workspace domain; use the public GitHub link if you do not have VIT access.

Public dataset ZIP (GitHub)

Google Drive dataset ZIP (institution-only)

Google Drive model artifacts (institution-only)